TEACHING WALLAH

Teaching Exam

Subject – Computer Science

Topic – DBMS

Lecture No.- 14

By- Rahul Sir

Topics to be Covered

Topic

1:- CARESTIAN PRODUCT
2-JOINS
3- NATURAL JOIN
4- INNER JOIN
5- DIFFERENCE BETWEEN NATURAL AND INNER JOIN
6->EQUI JOINS
7->Self Join
8->Left Outer Join
9-> Right Outer Join
10-> Full Outer Join
11-> TRANSACTION
12-> ACID PROPERTY

Topic : Cartesian Product

The Cartesian product in SQL is the result of combining every row from one table with every row from another. This operation is most often generated through a CROSS JOIN , forming a result set containing all possible pairs of rows from the two tables. While powerful, it can lead to very large result sets, especially when working with tables containing many rows, so it should be used with caution. In fact, I'll bet many developers probably first discover Cartesian products the hard way by accidentally omitting a join condition of some kind, only to end up with a massive result set.

Topic : Cartesian Product

TEACHING WALLAH

Degree:- Total Number Column
Cardenality:- Total Number Of Tuples

There is 2 Relation , R1 and R2 so Cartesian Product is R1XR2

It combines R1 and R2 without any condition. It is denoted by X.

Degree of R1 XR2 = degree of R1 + degree of R2

{degree = total no of columns}

Cardenality of R1 X R2 = Cardenality of R1 X Cardenality of R2
{Cardenality = multiply of total tuple in Relation R1 & R2

Topic : Cartesian Product

TEACHING WALLAH

R1 TABLE

ROLL	NAME
1	ROHAN SHARMA
2	RAHUL GUPTA
3	HARI OM

R2 TABLE

SALES YEAR	TARGET ACHEIVE
2024	4025
2025	7036

R1 X R2 TABLE **DEGREE=> 2+2=4** **CARDENALITY=> 3X2=6**

ROLL NO	NAME	SALES YEAR	TARGET ACHEIVE
1	ROHAN SHARMA	2024	4025
1	ROHAN SHARMA	2025	7036
2	RAHUL GUPTA	2024	4025
2	RAHUL GUPTA	2025	7036
3	HARI OM	2024	4025
3	HARI OM	2025	7036

Topic : Cartesian Product

TEACHING WALLAH

Table : Customers

ID	Name
1	Raj Mehta
2	Sanjay Mishra
3	Aditi Gupta

Table_Name : Shopping_Details

ID	Item_Name
2	Chips
3	Chocolate

Table : Result

3 x 2 = 6 rows

Customers CustID	Name	Shopping_details.CustID	Item_Name
1	Raj Mehta	2	Chips
1	Raj Mehta	3	Chocolate
2	Sanjay Mishra	2	Chips
2	Sanjay Mishra	3	Chocolate
3	Aditi Gupta	2	Chips
3	Aditi Gupta	3	Chocolate

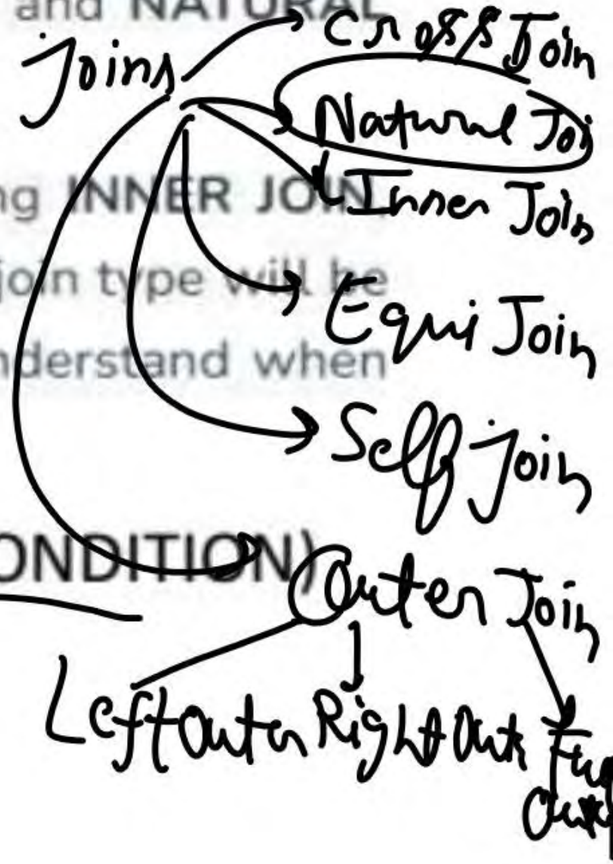
TEACHING WALLAH

Topic : SQL JOINS

SQL joins are fundamental tools for combining data from multiple tables in relational databases. Joins allow **efficient data retrieval**, which is essential for generating meaningful observations and solving **complex business queries**. Understanding SQL join types, such as **INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN**, **FULL JOIN**, and **NATURAL JOIN**, is critical for working with relational databases.

In this article, we will cover the **different types of SQL joins,** including **INNER JOIN**, **LEFT OUTER JOIN**, **RIGHT JOIN**, **FULL JOIN**, and **NATURAL JOIN**. Each join type will be explained with examples, **syntax**, and practical use cases to help us understand when and how to use these joins effectively.

- SQL JOINS = CROSS PRODUCT + SELECT STATEMENT (CONDITION)
- Cross Product is also known as Cartesian Product.



Topic : SQL JOINS

TEACHING WALLAH

What is SQL Join?

An SQL JOIN clause is used to **query** and **access data** from multiple tables by establishing **logical relationships** between them. It can access data from multiple tables simultaneously using common key values shared across different tables. We can use **SQL JOIN** with **multiple tables**. It can also be paired with other clauses, the most popular use will be using JOIN with WHERE clause to filter data retrieval.

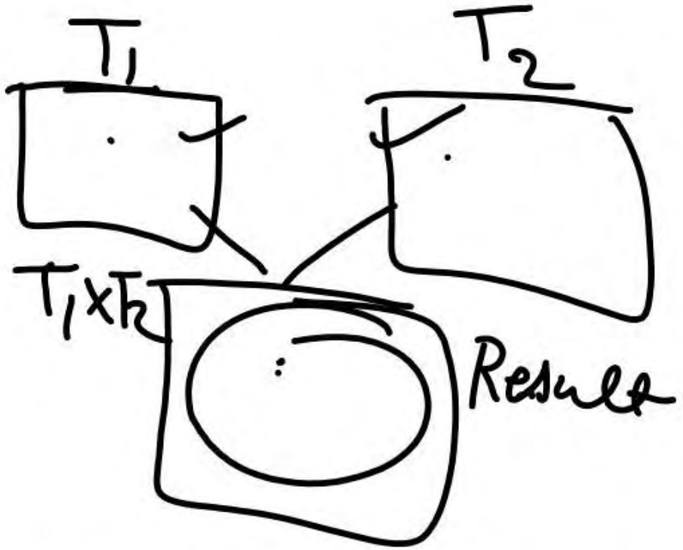
Before diving into the specifics, let's visualize how each join type operates:

- **INNER JOIN**: Returns only the rows where there is a match in both tables.
- **LEFT JOIN (LEFT OUTER JOIN)**: Returns all rows from the left table, and the matched rows from the right table. If there's no match, NULL values are returned for columns from the right table.
- **RIGHT JOIN (RIGHT OUTER JOIN)**: Returns all rows from the right table, and the matched rows from the left table. If there's no match, NULL values are returned for columns from the left table.
- **FULL JOIN (FULL OUTER JOIN)**: Returns all rows when there is a match in one of the tables. If there's no match, NULL values are returned for columns from the table without a match.

Topic : NATURAL JOINS

TEACHING WALLAH

A **Natural Join** performs a join between two tables based on columns that have the same name and compatible data types. This join operation automatically determines the common columns between the tables and removes duplicates in the result set. The resulting table will contain only one copy of the common columns, making the result concise and free from redundant data.



SYNTAX:-

Select * from Table1 Natural Joins Table2

Or

Select * From TABLE1,TABLE2 Where Table1.matching column = Table2.matching Column.

Topic : NATURAL JOINS

TEACHING WALLAH

Natural Joins:-

Find the Emp Names who is working in a department.

PK — Employee

E_NO	E_NAME	ADDRESS
1	RAM	DELHI
2	VARUN	MUMBAI
3	RAVI	KANPUR
4	AMRIT	DELHI

PK — Department — FK

DEPT_NO	NAME	E_NO
D1	HR	1
D2	IT	2
D3	MARKETING	3

Select E_NAME From Employee Natural Joins Department where Employee.E_NO = Department.E_NO

Subquery — Condition — PK — FK

Topic : NATURAL JOINS

TEACHING WALLAH

CARTESIAN PRODUCT:-

HIGHLITED SHOW THE RESULT

Result

RAM
Varun
Ravi

E_NO	E_NAME	DEPT_NO	E_NO
1	RAM	D1	1
1	RAM	D2	2
1	RAM	D3	3
2	VARUN	D1	1
2	VARUN	D2	2
2	VARUN	D3	3
3	RAVI	D1	1
3	RAVI	D2	2
3	RAVI	D3	3
4	AMRIT	D1	1
4	AMRIT	D2	2
4	AMRIT	D3	3

TEACHING WALLAH

Topic : INNER JOINS

1. SQL INNER JOIN

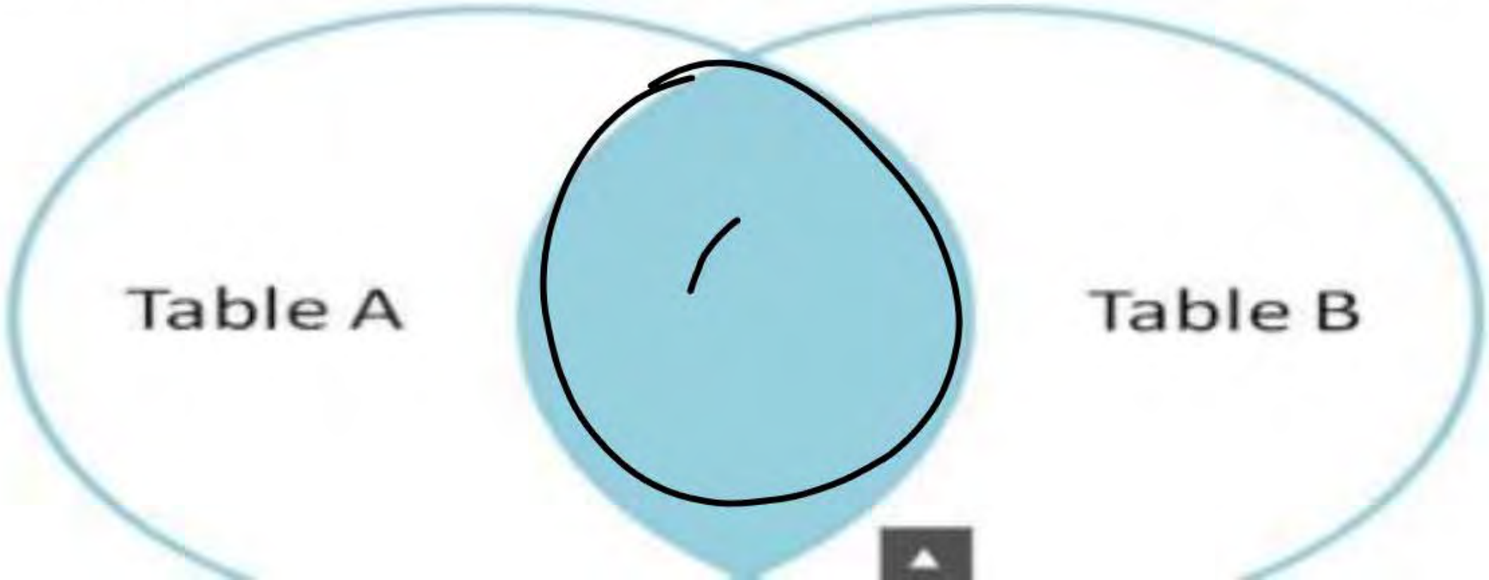
The INNER JOIN keyword selects all rows from both the tables as long as the condition is satisfied. This keyword will create the **result-set** by combining all rows from both the tables where the **condition satisfies** i.e value of the common field will be the same.

Syntax

table. column name

```
SELECT table1.column1,table1.column2,table2.column1,.... FROM table1 INNER
JOIN table2 ON table1.matching_column = table2.matching_column;
```

Note: We can also write JOIN instead of INNER JOIN. JOIN is same as INNER JOIN.



Topic : INNER JOINS

TEACHING WALLAH

Table_Name : Customers

CustID	Name	Phone_Number
1	Raj Mehta	98540XXXXX
2	Sanjay Mishra	88888XXXXX
3	Aditi Gupta	67809XXXXX
4	Manish Chopra	12345XXXXX

Table_Name: Shopping_Details

ItemID	CustID	Item_Name	Quantity
1	2	Chips	2
2	3	Chocolate	5
3	5	Dress	8

Primary key

Foreign key

```
Select Customers.Name, Shopping_Details.Item_Name, Shopping_Details.Quantity from
Customers Inner Join Shopping_Details On Customers.CustId = Shopping_Details.
                                                            CustID
```

Temporary Table

Customers CustID	Name	Phone_Number	Item_ID	Shopping_Details.CustID	Item_Name	Quantity
1	Raj Mehta	98540XXXXX	NULL	NULL	NULL	NULL
2	Sanjay Mishra	88888XXXXX	1	2	Chips	2
3	Aditi Gupta	67809XXXXX	2	3	Chocolate	5
4	Manish Chopra	12345XXXXX	NULL	NULL	NULL	NULL
NULL	NULL	NULL	3	5	Dress	8

Resultant Table

Name	Item_Name	Quantity
Sanjay Mishra	Chips	2
Aditi Gupta	Chocolate	5

Topic : Difference between INNER JOINS and NATURAL JOINS

TEACHING WALLAH

Difference Between InnerJoin & NaturalJoin

Following are the key differences between InnerJoin and NaturalJoin -

Criteria	Inner Join	Natural Join
Join Condition	Requires an explicit condition using the ON clause.	Automatically matches columns with the same name.
Column Selection	Requires the user to Specify which columns to compare.	Automatically detects columns with the same name.
Duplicate Columns	May return duplicate columns In the result set.	Does not return duplicate columns.
Control	Offers more control over the Join Operation.	Less control, as it depends on column names.
Risk Errors	Less risk if columns are explicitly specified.	Higher risk if unintended columns share the same name.

Topic : EQUI JOINS

1. EQUI JOIN :

EQUI JOIN creates a JOIN for equality or matching column(s) values of the relative tables. EQUI JOIN also create JOIN by using JOIN with ON and then providing the names of the columns with their relative tables to check equality using equal sign (=).

Syntax :

```
SELECT column_list
FROM table1, table2....
WHERE table1.column_name =
table2.column_name;
```

TEACHING WALLAH

Topic : EQUI JOINS

Equi Joins:-

Find the Emp Names who is working in a department having same location as their address.

E_NO	E_NAME	ADDRESS
1	RAM	DELHI
2	VARUN	MUMBAI
3	RAVI	KANPUR
4	AMRIT	DELHI

DEPT_NO	LOCATION	E_NO
D1	DELHI	1
D2	PUNE	2
D3	PATANA	3

Select E_name from Employee ,Department where Employee.E_NO = Department.E_NO and Employee.Address=Department.Location

Note:- Here Equi Joins looking like Natural Join along with equal Condition.

Topic : EQUI JOINS

TEACHING WALLAH

Equi Joins:-

Find the Emp Names who is working in a department having same location as their address.

E_NO	E_NAME	ADDRESS
1	RAM	DELHI
2	VARUN	MUMBAI
3	RAVI	KANPUR
4	AMRIT	DELHI

DEPT_NO	LOCATION	E_NO
D1	DELHI	1
D2	PUNE	2
D3	PATANA	3

(4)

Select E_name from Employee ,Department where Employee.E_NO = Department.E.NO and Employee.Address=Department.Location

Note:- Here Equi Joins looking like Natural Join along with equal Condition.

TEACHING WALLAH

Topic : EQUI JOINS

Highlighted show the output of Equi joins

E_NO	E_NAME	DEPT_NO	E_NO
1	RAM	DELHI	1
1	RAM	PUNE	2
1	RAM	PATANA	3
2	VARUN	DELHI	1
2	VARUN	PUNE	2
2	VARUN	PATANA	3
3	RAVI	DELHI	1
3	RAVI	PUNE	2
3	RAVI	PATANA	3
4	AMRIT	DELHI	1
4	AMRIT	PUNE	2
4	AMRIT	PATANA	3

Topic : SELF JOIN

TEACHING WALLAH

A Self Join is simply a regular **join operation** where a table is joined with itself. This allows us to compare rows within the **same table**, which is particularly useful when working with **hierarchical data** or when comparing related rows from a single table.

For example, a Self Join can help us retrieve employee-manager **relationships**, where each employee in the table has a reference to their manager's ID.

Syntax:

from table1, table2

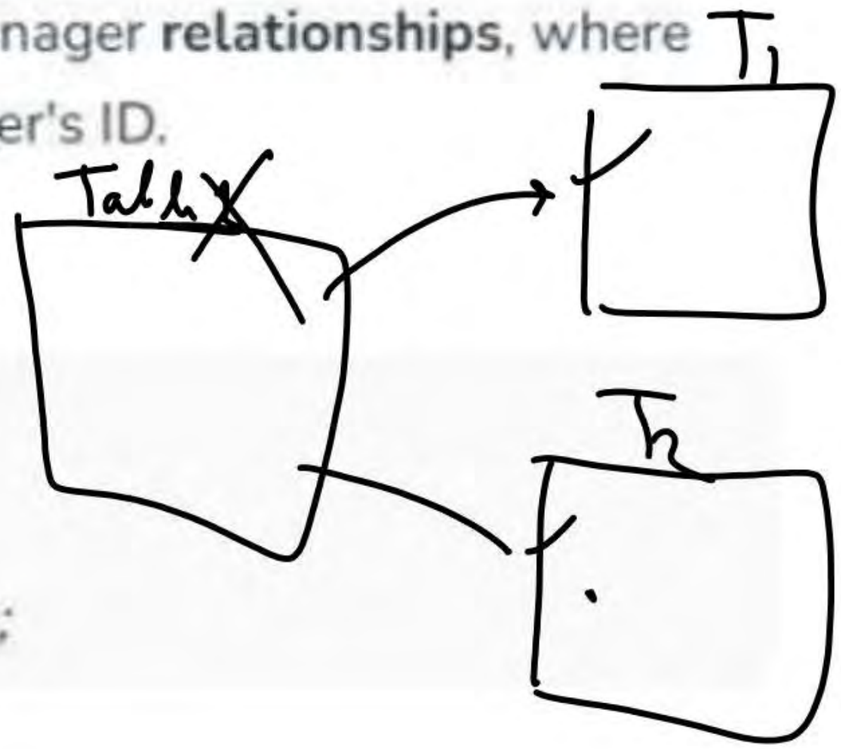
table1 alias T_1

table1 alias T_2

$T_1.col = T_2.col$

$R_1 \times R_1$

```
SELECT columns
FROM table AS alias1
JOIN table AS alias2 ON alias1.column = alias2.column;
```



Topic : SELF JOIN

TEACHING WALLAH

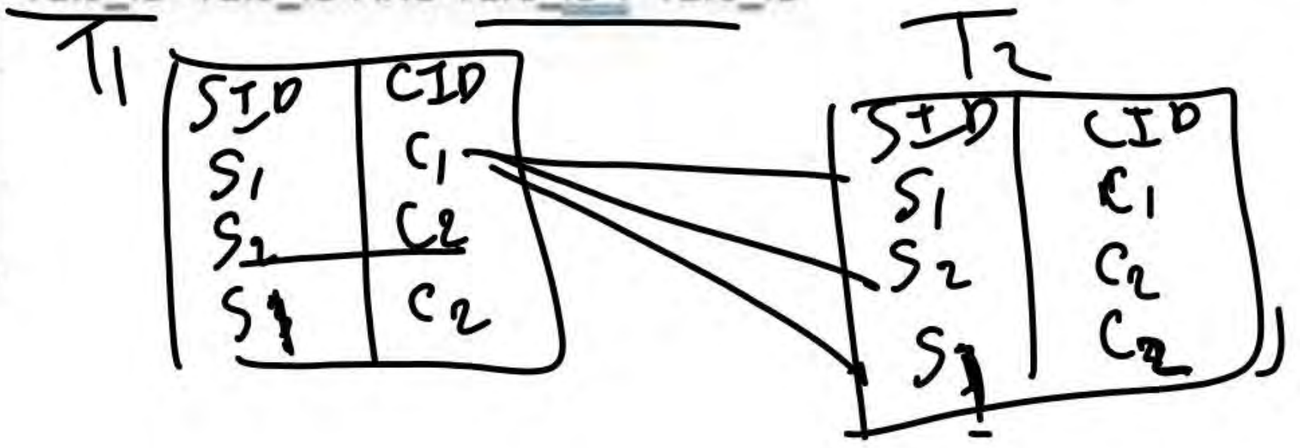
SELF JOIN

Find Student id who is enrolled in at least two Courses

Note:- JOINS= CROSS PRODUCT + CONDITION

S_ID	C_ID	SINCE
S1	C1	2016
S2	C2	2017
S1	C2	2017

SELECT S_ID,C_ID FROM STUDY AS T1, STUDY AS T2 WHERE T1.S_ID=T2.S_ID AND T1.C_ID !=T2.C_ID



Topic : SELF JOIN

S1	C1	S1	C1
S1	C1	S2	C2
S1	C1	S1	C2
S2	C2	S1	C1
S2	C2	S2	C2
S2	C2	S1	C2
S1	C2	S1	C1
S1	C2	S2	C2
S1	C2	S1	C2

Highlighted show the result of self join

Topic : LEFT OUTER JOIN

TEACHING WALLAH

2. SQL LEFT JOIN

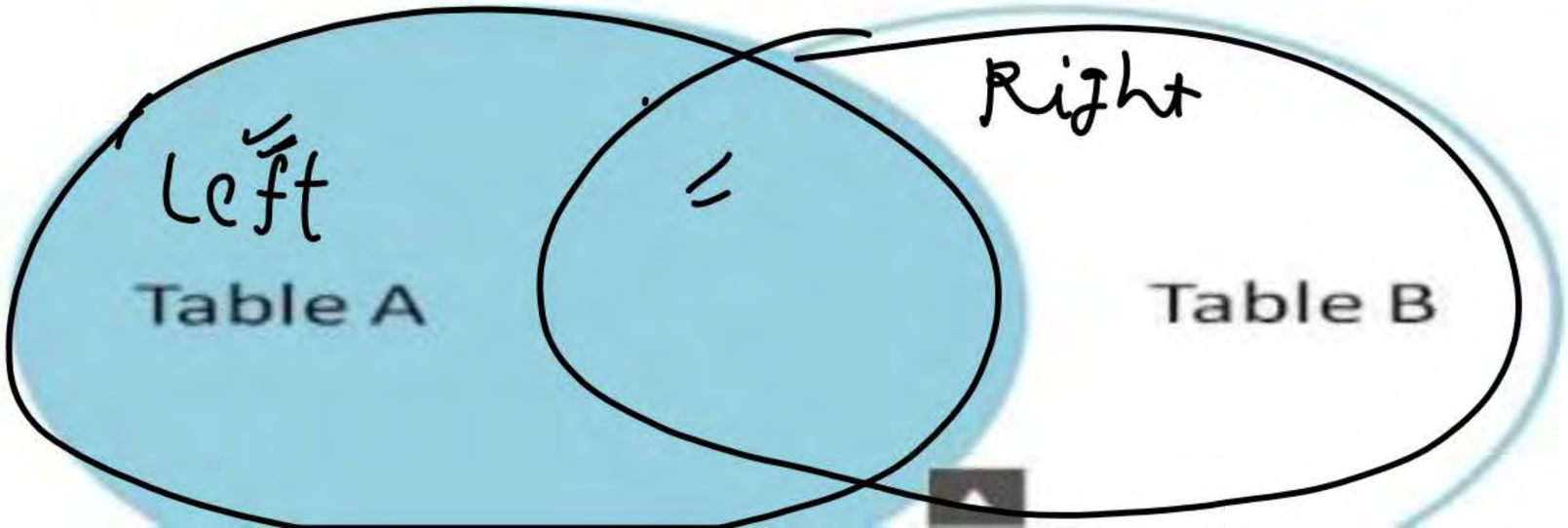
A **LEFT JOIN** returns all rows from the left table, along with matching rows from the right table. If there is no match, NULL values are returned for columns from the right table. LEFT JOIN is also known as **LEFT OUTER JOIN**.

Outer Join → Left Outer Join, Right Outer Join, Full Outer Join

Syntax

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
LEFT JOIN table2
ON table1.matching_column = table2.matching_column;
```

Note: We can also use LEFT OUTER JOIN instead of LEFT JOIN, both are the same.



Topic : LEFT OUTER JOIN

TEACHING WALLAH

LEFT OUTER JOIN

EMP

EMP_NO	E_NAME	DEPT_NO
E1	VARUN	D1
E2	AMIT	D2
E3	RAHUL	D1
E4	SURESH	NULL

DEPT

DEPT_NO	D_NAME	LOCATION
D1	IT	DELHI
D2	HR	HYDRABAD
D3	FINANCE	PUNE

Select EMP_NO, E_NAME,D_NAME,LOCATION FROM EMP LEFT OUTER JOIN DEPT ON EMP.DEPT_NO=DEPT.DEPT_NO

Topic : LEFT OUTER JOIN

TEACHING WALLAH

EMP_NO	E_NAME	D_NAME	LOCATION
E1	VARUN	IT	DELHI
E2	AMIT	HR	HYDRABAD
E3	RAHUL	IT	DELHI
E4	SURESH	NULL	NULL

Right (Common Data with Left Side Table)

LEFT OUTER JOIN= NATURAL JOIN + REMAINING LEFT SIDE TABLE DATA

Topic : RIGHT OUTER JOIN

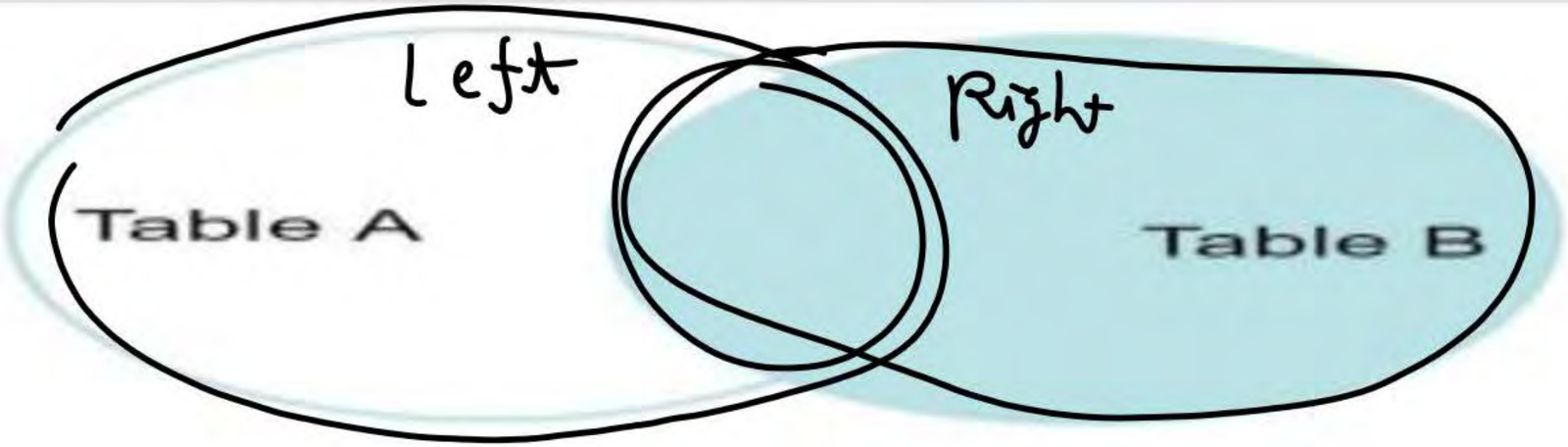
TEACHING WALLAH

3. SQL RIGHT JOIN

RIGHT JOIN returns all the rows of the table on the **right side of the join** and matching rows for the table on the left side of the join. It is very similar to **LEFT JOIN** for the rows for which there is no matching row on the left side, the result-set will contain **null**. **RIGHT JOIN** is also known as **RIGHT OUTER JOIN**.

Syntax

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
RIGHT JOIN table2
ON table1.matching_column = table2.matching_column;
```



Topic : RIGHT OUTER JOIN

TEACHING WALLAH

RIGHT OUTER JOIN

EMP

EMP_NO	E_NAME	DEPT_NO
E1	VARUN	D1
E2	AMIT	D2
E3	RAHUL	D1
NULL	NULL	NULL

DEPT

DEPT_NO	D_NAME	LOCATION
D1	IT	DELHI
D2	HR	HYDRABAD
D3	FINANCE	PUNE
D4	TESTING	NOIDA

Select EMP_NO, E_NAME,D_NAME,LOCATION FROM EMP RIGHT OUTER JOIN DEPT ON EMP.DEPT_NO=DEPT.DEPT_NO

Topic : RIGHT OUTER JOIN

TEACHING WALLAH

EMP_NO	E_NAME	D_NAME	LOCATION
E1	VARUN	IT	DELHI
E2	AMIT	HR	HYDRABAD
E3	RAHUL	IT	DELHI
NULL	NULL	TESTING	NOIDA
NULL	NULL	Finance	Pune

LeftSide Common Data with Right table

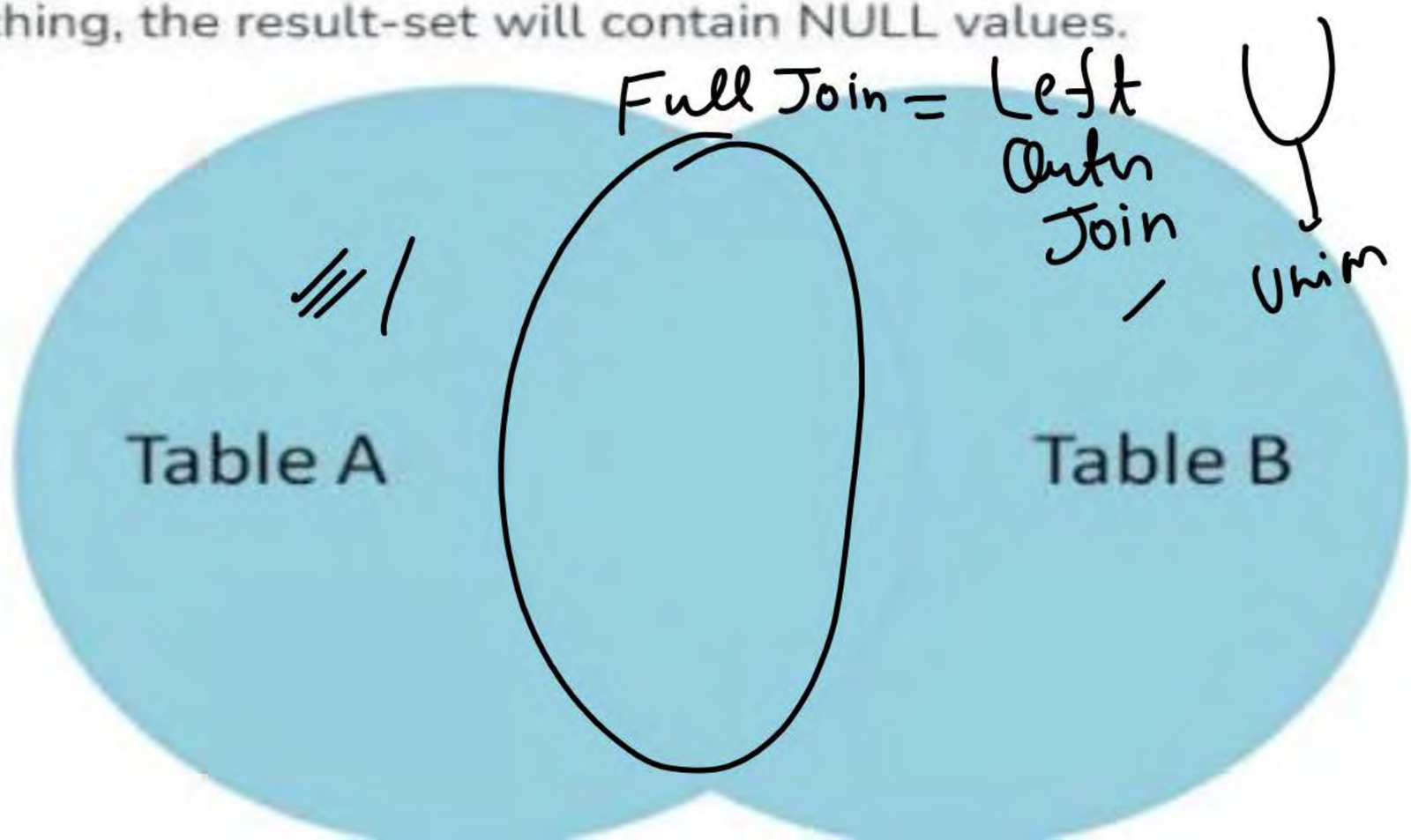
RIGHT OUTER JOIN = NATURAL JOIN + REMAINING ~~RIGHT SIDE~~ TABLE DATA

Topic : FULL OUTER JOIN

TEACHING WALLAH

4. SQL FULL JOIN

FULL JOIN creates the result-set by combining results of both **LEFT JOIN** and **RIGHT JOIN**. The result-set will contain all the rows from both tables. For the rows for which there is no matching, the result-set will contain NULL values.



Topic : FULL OUTER JOIN

TEACHING WALLAH

FULL OUTER JOIN

EMP

EMP_NO	E_NAME	DEPT_NO
E1	VARUN	D1
E2	AMIT	D2
E3	RAHUL	D1

DEPT

DEPT_NO	D_NAME	LOCATION
D1	IT	DELHI
D2	HR	HYDRABAD
D3	FINANCE	PUNE
D4	TESTING	NOIDA

```
Select EMP_NO, E_NAME,D_NAME,LOCATION FROM EMP FULL OUTER JOIN DEPT ON
EMP.DEPT_NO=DEPT.DEPT_NO
```

Topic : FULL OUTER JOIN

TEACHING WALLAH

EMP_NO	E_NAME	D_NAME	LOCATION
E1	VARUN	IT	DELHI
E2	AMIT	HR	HYDRABAD
E3	RAHUL	IT	DELHI
NULL	NULL	TESTING	NOIDA
NULL	NULL	FINANCE	PUNE

FULL OUTER JOIN = LEFT TABLE (UNION) RIGHT TABLE

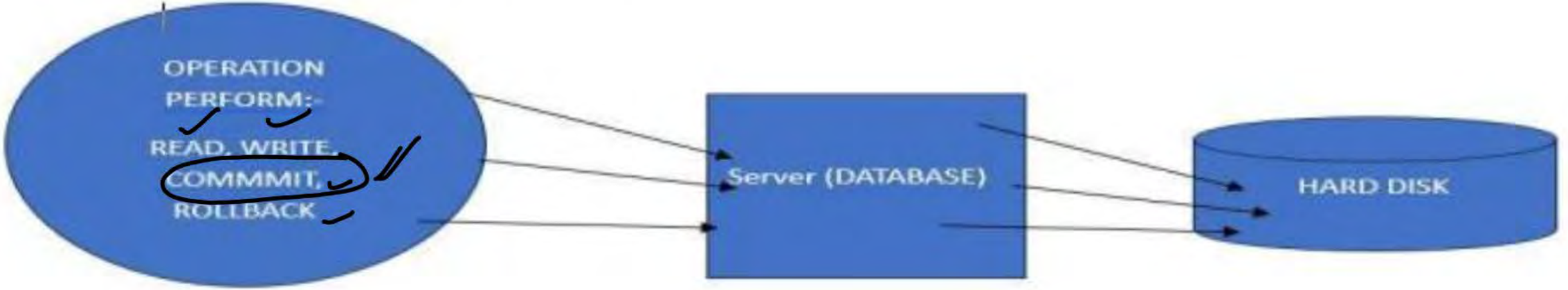
ALL THE COMPLETE DATA OF BOTH LEFT AND RIGHT TABLE

Topic : TRANSCATION

TEACHING WALLAH

What is Transaction in DBMS?

It is an action or sequence of actions passed out by a single user and/or application program that reads or updates the contents of the database. A transaction is a logical piece of work of any database, which may be a complete program, a fraction of a program, or a single command (like the: SQL command INSERT or UPDATE) that may involve any number of processes on the database. From the database point of view, the implementation of an application program can be considered as one or more transactions with non-database processing working in between.



Topic : TRANSCATION

TEACHING WALLAH

R(A) ->1000

A=A-500

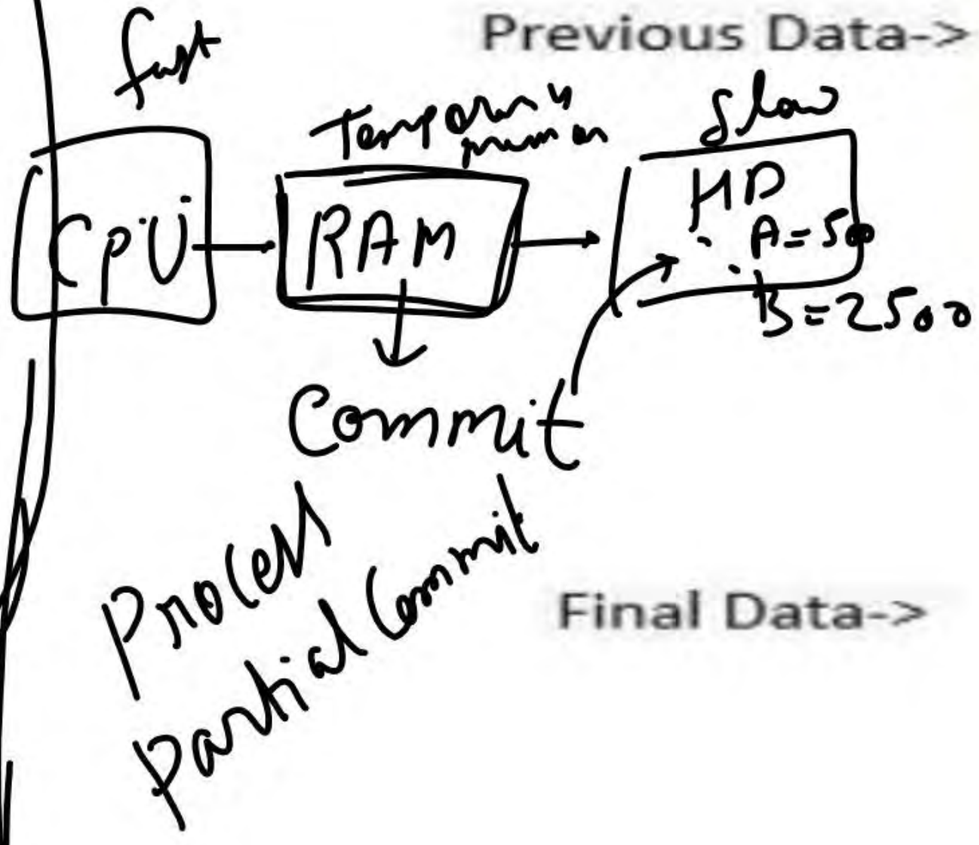
W(A)->500

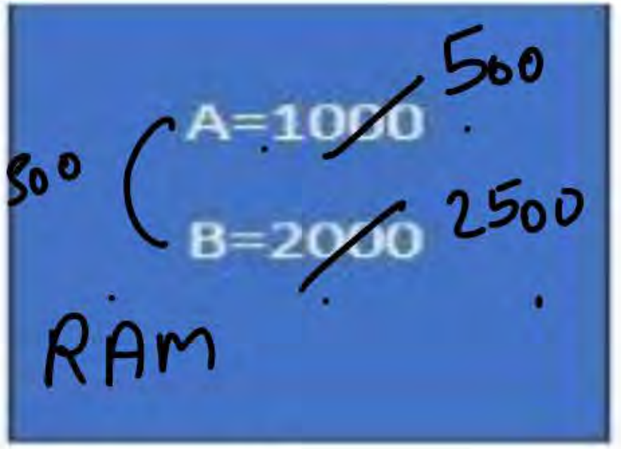
R(B)->2000

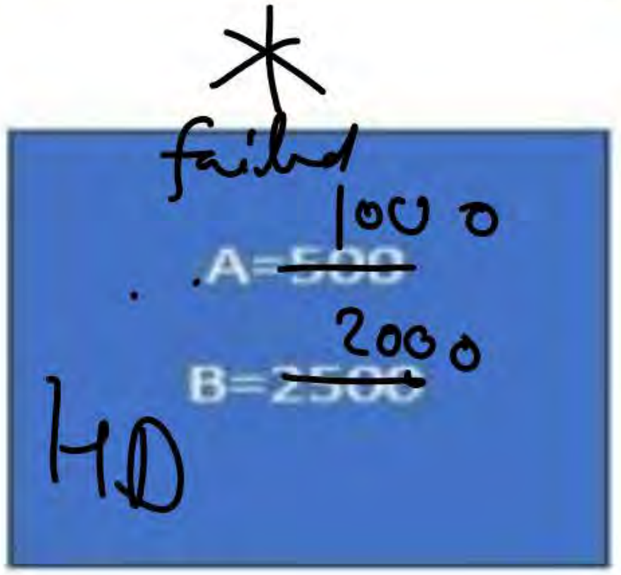
B=B+500

W(B) -> 2500

COMMIT





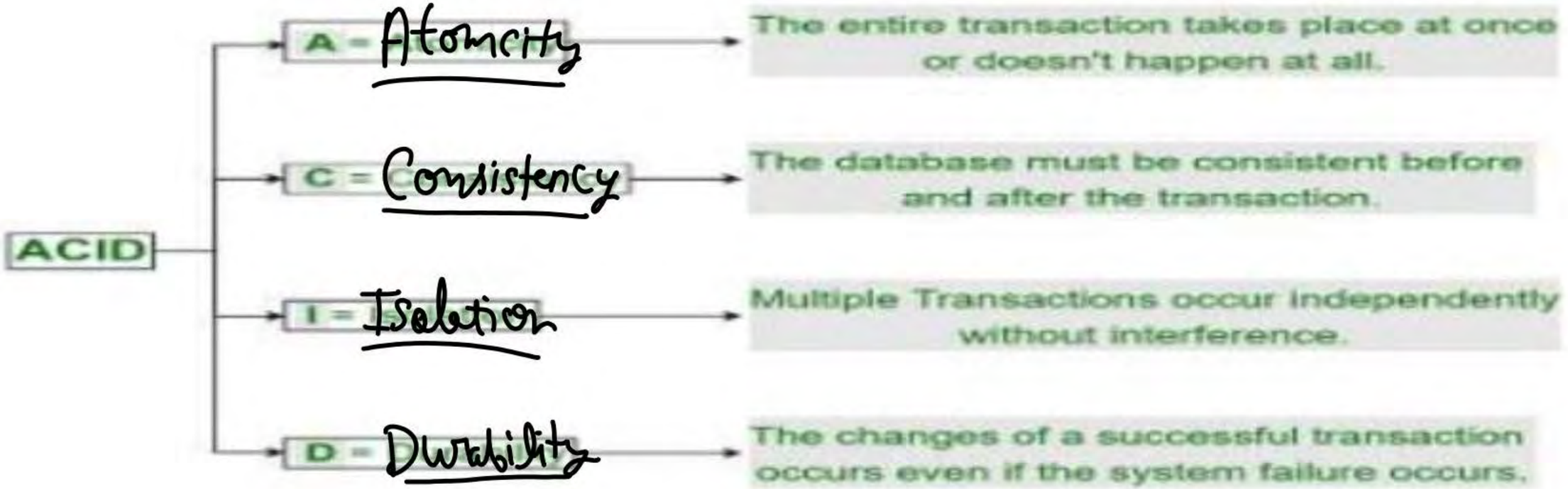


Topic : ACID PROPERTY IN TRANSACTION

TEACHING WALLAH

ACID stands for **Atomicity**, **Consistency**, **Isolation**, and **Durability**. These four key properties define how a transaction should be processed in a reliable and predictable manner, ensuring that the database remains consistent, even in cases of failures or concurrent accesses.

ACID Properties in DBMS



Topic : ATOMICITY

1. Atomicity: "All or Nothing"

Atomicity ensures that a transaction is **atomic**, it means that either the entire transaction completes fully or doesn't execute at all. There is no in-between state i.e. transactions do not occur partially. If a transaction has multiple operations, and one of them fails, the whole transaction is rolled back, leaving the database unchanged. This avoids partial updates that can lead to inconsistency.

- **Commit**: If the transaction is successful, the changes are permanently applied.
- **Abort/Rollback**: If the transaction fails, any changes made during the transaction are discarded.

Topic : ATOMICITY

TEACHING WALLAH

Example: Consider the following transaction T consisting of **T1** and **T2** : Transfer of $100 from account **X** to account **Y** .

Before: X : 500	Y : 200
Transaction T	
T1	T2
Read (X)	Read (Y)
X: = X - 100	Y: = Y + 100
Write (X)	Write (Y)
After: X : 400	Y : 300

Example

If the transaction fails after completion of T1 but before completion of T2 , the database would be left in an inconsistent state. With Atomicity, any part of the transaction fails, the entire process is rolled back to its original state, and no partial changes are made.

2. Consistency: Maintaining Valid Data States

Consistency ensures that a database remains in a valid state before and after a transaction. It guarantees that any transaction will take the database from one consistent state to another, maintaining the rules and constraints defined for the data. In simple terms, a transaction should only take the database from one **valid** state to another. If a transaction violates any database rules or constraints, it should be rejected, ensuring that only consistent data exists after the transaction.

Topic : CONSISTENCY

TEACHING WALLAH

Example: Suppose the sum of all balances in a bank system should always be constant. Before a transfer, the total balance is **$700**. After the transaction, the total balance should remain $700. If the transaction fails in the middle (like updating one account but not the other), the system should maintain its consistency by rolling back the transaction

Total before T occurs = 500 + 200 = 700 .

Total after T occurs = 400 + 300 = 700 .

X = 500 Rs | Y = 500 Rs

T	T''
Read (X) X :=X*100 Write (X) Read (Y) Y:= Y - 50 Write (Y)	Read (X) Read (Y) Z:= X + Y Write (Z)

Topic : ISOLATION

TEACHING WALLAH

3. Isolation: Ensuring Concurrent Transactions Don't Interfere

This property ensures that **multiple transactions** can occur concurrently without leading to the **inconsistency** of the database state. Transactions occur independently without interference. Changes occurring in a particular transaction will not be visible to any other transaction until that particular change in that transaction is written to memory or has been committed.

This property ensures that when multiple transactions run at the same time, the result will be the same as if they were run one after another in a specific order. This property prevents issues such as **dirty reads** (reading uncommitted data), **non-repeatable reads** (data changing between two reads in a transaction), and **phantom reads** (new rows appearing in a result set after the transaction starts).

Topic : ISOLATION

Example: Let's consider two transactions:Consider two transactions T and T''.

- X = 500, Y = 500

X = 500 Rs	Y = 500 Rs
T	T''
Read (X) X :=X*100 Write (X) Read (Y) Y:= Y - 50 Write (Y)	Read (X) Read (Y) Z:= X + Y Write (Z)

Transaction T:

- T wants to transfer $50 from X to Y.
- T reads Y (value: 500), deducts $50 from X (new X = **450**), and adds $50 to Y (new Y = **550**).

Topic : ISOLATION

Transaction T'':

- **T''** starts and reads **X** (value: 500) and **Y** (value: 500), then calculates the sum: **500 + 500 = 1000**.

But, by the time **T** finishes, **X** and **Y** have changed to **450** and **550** respectively, so the correct sum should be **450 + 550 = 1000**. **Isolation** ensures that **T''** should not see the old values of **X** and **Y** while **T** is still in progress. Both transactions should be independent, and **T''** should only see the final state after **T** commits. This prevents inconsistent data like the incorrect sum calculated by **T''**

Topic : ISOLATION

Transaction T'':

- **T''** starts and reads **X** (value: 500) and **Y** (value: 500), then calculates the sum: **500 + 500 = 1000**.

But, by the time **T** finishes, **X** and **Y** have changed to **450** and **550** respectively, so the correct sum should be **450 + 550 = 1000**. **Isolation** ensures that **T''** should not see the old values of **X** and **Y** while **T** is still in progress. Both transactions should be independent, and **T''** should only see the final state after **T** commits. This prevents inconsistent data like the incorrect sum calculated by **T''**

Topic : DURABILITY

4. Durability: Persisting Changes

This property ensures that once the transaction has completed execution, the updates and modifications to the database are stored in and written to disk and they persist even if a system failure occurs. These updates now become permanent and are stored in **non-volatile memory**. In the event of a failure, the DBMS can recover the database to the state it was in after the last committed transaction, ensuring that no data is lost.

Example: After successfully transferring money from Account A to Account B, the changes are stored on disk. Even if there is a crash immediately after the commit, the transfer details will still be intact when the system recovers, ensuring durability.

THANK YOU